

Choice/switch и наследование/полиморфизм: два способа моделировать варианты

Экспрешн-проблема (expression problem) в языках программирования и в XSD

Август 2026

- [Введение](#)
- [1. Формальная взаимокодируемость](#)
 - [1.1. От switch к полиморфизму: паттерн Visitor](#)
 - [1.2. От полиморфизма к switch: Черч-кодирование](#)
- [2. Где эквивалентность заканчивается: expression problem](#)
 - [2.1. switch / choice : закрытый набор вариантов, открытый набор операций](#)
 - [2.2. Наследование / полиморфизм: открытый набор вариантов, закрытый набор операций](#)
 - [2.3. Сводная таблица](#)
- [3. Тот же выбор — в XSD](#)
 - [3.1. xs:choice — аналог switch, закрытый список](#)
 - [3.2. xsi:type + наследование — аналог полиморфизма, открытый список](#)
 - [3.3. Почему в разбираемой схеме выбран choice](#)
- [4. Практические критерии выбора](#)
- [5. Промежуточные решения](#)
- [Заключение](#)

Введение

На первый взгляд может показаться, что наследование и полиморфизм в объектно-ориентированном программировании — это просто более многословный, «синтаксически подслащённый» способ сделать то же самое, что делает обычный `choice / switch / match`: выбрать одно из нескольких известных поведений в зависимости от варианта данных.

Эта лекция разбирает, почему это не совсем так. Оба механизма действительно взаимокодируемы — в теоретико-вычислительном смысле они эквивалентны. Но у них принципиально разное **направление расширяемости**, и это различие формализовано в теории языков программирования под названием **expression problem** («экспрешн-проблема»). Понимание этого различия объясняет, почему оба механизма продолжают сосуществовать в языках программирования, а не вытесняют друг друга, и почему в языках схем данных (таких как XSD) присутствуют оба аналога — `xs:choice` и `xsi:type`/наследование типов.

1. Формальная взаимокодируемость

Начнём с того, в чём интуиция про «синтаксический сахар» действительно верна: на уровне вычислительной модели `switch`-подобная конструкция и полиморфная диспетчеризация через виртуальные методы **эквивалентны** — любую программу, написанную в одном стиле, можно механически переписать в другой.

1.1. От `switch` к полиморфизму: паттерн **Visitor**

Классический пример — паттерн **Visitor** в ООП. Он буквально представляет собой способ эмулировать `switch` по типу через двойную виртуальную диспетчеризацию:

```
interface Shape {
    double area();
}
class Circle implements Shape {
    double r;
    public double area() { return Math.PI * r * r; }
}
class Square implements Shape {
    double s;
    public double area() { return s * s; }
}
```

Это и есть перевод `switch (shape.type) { case CIRCLE: ...; case SQUARE: ...; }` на язык виртуальных методов: вместо централизованной таблицы веток каждый класс «знает» свою собственную ветку.

1.2. От полиморфизма к `switch`: Черч-кодирование

Обратное преобразование тоже существует и известно как **кодирование Чёрча** (Church encoding): любой объект с виртуальными методами можно представить как замыкание, хранящее «внутри себя», какая у него метка варианта, и диспетчеризирующее вызовы через `switch`:

```
def make_circle(r):
    def area():
        return math.pi * r ** 2
    return {"area": area}

def make_square(s):
    def area():
        return s ** 2
    return {"area": area}
```

Оба фрагмента вычислительно эквивалентны своим ООП-версиям. Это подтверждает: с точки зрения «что можно вычислить», `choice / switch` и наследование/полиморфизм — два способа выразить одну и ту же вещь: *выполнить разное поведение в зависимости от того, какой перед нами вариант данных*.

2. Где эквивалентность заканчивается: expression problem

Формальная взаимокодируемость не означает практическую взаимозаменяемость. Разница проявляется, когда встаёт вопрос: **что вы будете расширять чаще — набор вариантов данных или набор операций над ними?**

Эта дилемма формализована Филипом Уодлером (Philip Wadler) под названием **expression problem**: постройте такую систему типов данных и операций над ними, чтобы можно было добавлять **и новые варианты данных, и новые операции**, не переписывая существующий код и сохраняя типобезопасность. Оказывается, ни `switch`-подход, ни классическое ООП-наследование не решают обе стороны проблемы одновременно — каждый хорош лишь в одном направлении.

2.1. `switch` / `choice`: закрытый набор вариантов, открытый набор операций

При подходе на основе `switch` все варианты перечислены **в одном месте** — в определении типа (`enum` , `union` , `choice`). Это делает:

- **добавление новой операции — лёгким**: пишете новую функцию с одним `switch` , все случаи видны сразу в одном месте, компилятор часто может проверить полноту перебора (exhaustiveness check);
- **добавление нового варианта — тяжёлым**: нужно найти и поправить *каждый* `switch` по этому типу во всей кодовой базе — рискуя пропустить один из них.

```
def area(shape):
    match shape:
        case Circle(r): return math.pi * r ** 2
        case Square(s): return s ** 2
        # добавление Triangle требует найти и обновить
        # BCE такие match-выражения в проекте
```

2.2. Наследование / полиморфизм: открытый набор вариантов, закрытый набор операций

При ООП-подходе набор операций фиксируется **интерфейсом** (абстрактным классом), а каждый вариант реализует его самостоятельно. Это делает:

- **добавление нового варианта — лёгким**: просто новый класс, реализующий интерфейс, ничего в существующем коде трогать не нужно;
- **добавление новой операции — тяжёлым**: нужно добавить метод в интерфейс и реализовать его в *каждом* существующем классе-наследнике.

```
class Shape(ABC):
    @abstractmethod
    def area(self): ...

class Circle(Shape):
    def area(self): return math.pi * self.r ** 2

class Square(Shape):
    def area(self): return self.s ** 2

# Triangle(Shape) — просто новый класс, ничего чужого не трогаем.
# А вот добавить метод perimeter() — придётся лезть
# в КАЖДЫЙ существующий класс-наследник.
```

2.3. Сводная таблица

	switch / choice / pattern matching	наследование / полиморфизм
Где перечислены варианты	В одном месте — определении типа	Разбросаны по классам-наследникам
Добавить новый вариант	Тяжело — искать все switch	Легко — новый класс
Добавить новую операцию	Легко — новая функция с одним switch	Тяжело — метод в каждом классе
Набор вариантов	Закрытый (фиксирован при определении типа)	Открытый (расширяем позже, даже сторонним кодом)
Проверка полноты компилятором	Часто да (exhaustiveness check)	Обычно нет

3. Тот же выбор — в XSD

Ровно та же дилемма — закрытый список против открытого — существует и в языках описания данных, включая XSD. Мы разбирали два механизма на примере схемы ON_ZAKZVG0_UserContract :

3.1. xs:choice — аналог switch, закрытый список

```
<xs:complexType name="ExtendedOrganizationInfoLogistics">
  <xs:choice>
    <xs:element name="OrganizationDetails" type="ExtendedOrganizationDetailsLogistics" />
    <xs:element name="OrganizationReference" type="ExtendedOrganizationReferenceLogistics" />
  </xs:choice>
</xs:complexType>
```

Список вариантов («ручной ввод реквизитов» или «ссылка на организацию») зафиксирован раз и навсегда прямо в этой схеме. Сторонняя система не может внести третий вариант, не переиздав саму схему .xsd — набор закрыт.

3.2. xsi:type + наследование — аналог полиморфизма, открытый список

```
<xs:complexType name="Animal" abstract="true">
  <xs:sequence>
    <xs:element name="Name" type="xs:string"/>
  </xs:sequence>
</xs:complexType>

<xs:complexType name="Dog">
  <xs:complexContent>
    <xs:extension base="Animal">
      <xs:sequence><xs:element name="Breed" type="xs:string"/></xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

```
</xs:extension>
</xs:complexContent>
</xs:complexType>
```

```
<Pet xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:type="Dog">
  <Name>Рекс</Name>
  <Breed>Овчарка</Breed>
</Pet>
```

Здесь — противоположная ситуация: кто угодно, имея доступ к базовому абстрактному типу `Animal`, может в **своей собственной**, отдельно опубликованной схеме объявить `Cat extends Animal`, и документы с `xsi:type="Cat"` будут валидны против оригинальной схемы автора, даже если тот о `Cat` никогда не знал. Список вариантов открыт для расширения третьими сторонами. Похожим образом работает и `substitutionGroup` — но на уровне элементов, а не типов.

3.3. Почему в разбираемой схеме выбран `choice`

Разработчики схемы `ON_ZAKZVG0_UserContract`, судя по её структуре, сознательно выбрали `xs:choice` там, где список вариантов организации (ручной ввод реквизитов vs. ссылка на уже существующую в системе организацию) — заведомо закрытый и контролируется одной стороной (оператором системы логистики). Здесь нет практической необходимости позволять третьим лицам добавлять новые «способы описать организацию» — а значит, `choice` выигрывает у `xsi:type` по всем практическим критериям: он проще для генерации XML, не требует пространства имён `xsi`, и полнота вариантов проверяется одним взглядом на схему.

4. Практические критерии выбора

Обобщая — вот вопросы, которые стоит задать при выборе подхода (одинаково применимо и к языкам программирования, и к схемам данных):

Выбирайте `switch` / `choice` / закрытый список, если: - набор вариантов стабилен и контролируется одной стороной; - операций над вариантами будет много и они будут часто добавляться; - важна проверка компилятором/валидатором полноты перебора всех вариантов; - простота и явность важнее расширяемости третьими сторонами.

Выбирайте наследование / полиморфизм / открытый список, если: - набор вариантов будет расти, в том числе усилиями сторонних разработчиков/систем; - набор операций (интерфейс) стабилен и меняется редко; - система должна оставаться расширяемой без модификации существующего кода/схемы (принцип открытости-закрытости, ОЦР).

5. Промежуточные решения

Стоит упомянуть, что современные языки и подходы пытаются смягчить дилемму *expression problem*, а не просто выбирать одну из сторон:

- **Тайп-классы** (Haskell) и **трейты** (Rust) позволяют добавлять и новые типы, и новые операции

независимо, если заранее спроектировать код соответствующим образом (реализация операции для типа может быть определена отдельно от обоих).

- **Расширяемые визитеры** (в духе “Object Algebras” или паттерна “Expression Problem solutions” в Scala) комбинируют открытость по обоим направлениям ценой дополнительной сложности.
- В XSD прямого аналога таким решениям нет — но можно комбинировать: например, использовать `xs:choice` с одним из вариантов, который сам по себе является `abstract`-типом, допускающим `xsi:type`-расширение — получая частичную открытость там, где она нужна, и закрытость там, где она предпочтительна.

Заключение

Наследование/полиморфизм — не «синтаксический сахар» над `choice / switch`, а инструмент с противоположным профилем расширяемости. Оба механизма вычислительно эквивалентны (что доказывается взаимной кодируемостью через Visitor и Church-кодирование), но на практике выбор между ними определяется не удобством записи, а тем, **какая сторона системы — набор вариантов или набор операций — будет меняться чаще и кем**. Экспрешн-проблема формализует это как фундаментальное противоречие, которое ни один из двух классических подходов не решает полностью сам по себе. Ровно то же соображение объясняет сосуществование `xs:choice` и `xsi:type`/наследования типов в XSD: это не дублирующие друг друга механизмы, а два инструмента, оптимальных для разных сценариев расширяемости схемы.